DBMS: MODULE-1

▼ Introduction

▼ Database System Applications

1. **Railway Reservation System** –In the rail route reservation framework, the information base is needed to store the record or information of ticket appointments, status about train's appearance, and flight. Additionally, if trains get late, individuals become acquainted with it through the information base update.
2. **Library Management System** –There are lots of books in the library so; it is difficult to store the record of the relative multitude of books in a register or duplicate. Along these lines, the data set administration framework (DBMS) is utilized to keep up all the data identified with the name of the book, issue date, accessibility of the book, and its writer.
3. **Banking** –Database the executive's framework is utilized to store the exchange data of the client in the information base.
4. **Education Sector** –Presently, assessments are led online by numerous schools and colleges. They deal with all assessment information through the data set administration framework (DBMS). In spite of that understudy's enlistments subtleties, grades, courses, expense, participation, results, and so forth all the data is put away in the information base.
5. **Credit card exchanges** – The database Management framework is utilized for buying on charge cards and age of month to month proclamations.
6. **Social Media Sites** –We all utilization of online media sites to associate with companions and to impart our perspectives to the world. Every day, many people group pursue these online media accounts like Pinterest, Facebook, Twitter, and Google in addition to. By the utilization of the data set administration framework, all the data of clients are put away in the information base and, we become ready to interface with others.
7. **Broadcast communications** – Without DBMS any media transmission organization can't think. The Database the executive's framework is fundamental for these organizations to store the call subtleties and month to month postpaid bills in the information base.
8. **Account** –The information base administration framework is utilized for putting away data about deals, holding and acquisition of monetary instruments, for example, stocks and bonds in a data set.
9. **Online Shopping** – These days, web-based shopping has become a major pattern. Nobody needs to visit the shop and burn through their time. Everybody needs to shop through web based shopping sites, (for example, Amazon, Flipkart, Snapdeal) from home. So all the items

are sold and added uniquely with the assistance of the information base administration framework (DBMS). Receipt charges, installments, buy data these are finished with the assistance of DBMS.

10. **Human Resource Management –** Big firms or organizations have numerous specialists or representatives working under them. They store data about worker's compensation, assessment, and work with the assistance of an information base administration framework (DBMS).
11. **Manufacturing –** Manufacturing organizations make various kinds of items and deal them consistently. To keep the data about their items like bills, acquisition of the item, amount, inventory network the executives, information base administration framework (DBMS) is utilized.
12. **Airline Reservation System –** This framework is equivalent to the railroad reservation framework. This framework additionally utilizes an information base administration framework to store the records of flight takeoff, appearance, and defer status.

▼ Purpose of database system

It is a collection of tools that enable users to create and manage databases. In other words, it is general-purpose software that allows users to create, manipulate, and design databases for a number of purposes.

Database systems are design to deal with large volumes of data. Data management comprises both the construction of data storage systems and the provision of data manipulation methods. Furthermore, the database system must maintain the security of the information held despite system crashes or attempts at unauthorised access. The system must avoid any unexpected effects if data is to be shared across multiple users.

The database applications were built on top of the file system.

The goal of a database management system (DBMS) is to transform the following:

1. Data into information.
2. Information into knowledge.
3. Knowledge of the action.

Characteristics of DBMS

- It manages and stores information in a server-based digital repository.
- Secondly, It can logically and visibly represent the data transformation process.
- Automatic backup and recovery techniques are built into the database management system.
- It has ACID features, which ensure that data is safe even if the system fails.
- It has the ability to make complex data connections more understandable.
- It's utilise to help with data manipulation and processing.
- It is utilise to keep information safe.
- It can examine the database from a variety of perspectives, depending on the needs of the user.

Advantages of DBMS

- Because it saves all of the data in a single database file and that record data is saves in the database, it can control database redundancy.

- Data sharing: Authorised users of a database management system can share data with a large number of other users.
- The database system is relatively easy to maintain due to its centralised architecture.
- It saves time by lowering the time it takes to create a product as well as the time it takes to maintain it.
- Backup: It consists of backup and recovery subsystems that create automatic data backups in the case of hardware or software failures and restore the data if necessary.
- It offers graphical user interfaces and application programme interfaces, among other options.

Disadvantages of DBMS

Hardware and software costs: To operate DBMS software, you'll need a fast data processor and a lot of memory.

Size: To run them efficiently, it takes up a lot of disc space and RAM.

Complexity: The database system adds to the complexity and demands.

Failure has a greater impact on the database since most organisations keep all of their data in a single database, and if the database is damage due to an electric outage or database corruption, the data might lost permanently.

▼ View of Data

https://youtu.be/I_PrZ1NHZr8

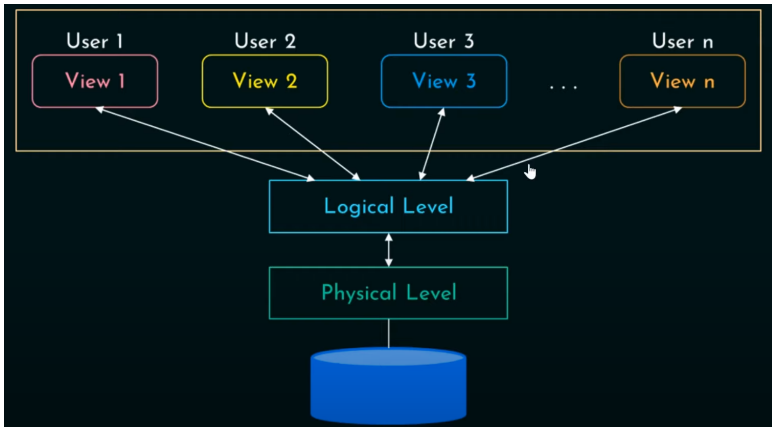
Why do we need multiple view of data ?

Ans The Multiple view of data maintains **Data Abstraction.**

Data Abstraction

1. Primary goal: **store** and **retrieve.**
2. **Convenient** and **efficient.**
3. **complex** data structure
4. **Hiding** the complexity.

Levels of Abstraction



▼ **Physical Level**

- Lowest level of abstraction.
- Deals with how the data are stored.
- Complex low level data structures.
- Database level.
- Storage level.

▼ **Logical Level**

- Deals with relationship.
- Entire database with simple data structures.
- Physical Data Independence.
- Database Administrator decides the logic of the level.

▼ **View Level**

- Highest level of abstraction.
- This level deals with the users and their previlaged access.
- This level deals with user interactions with the system.
- Have GUI in most cases..
- This level interact with the next Logical level where application program are retained.
- Several Views with ,particular security for each

▼ Database Languages

▼ **D**ata **D**efenition **L**anguages (**DDL**)

- **DDL** stands for **D**ata **D**efinition **L**anguage. It is used to define database structure or pattern.
- It is used to create schema, tables, indexes, constraints, etc. in the database.
- Using the DDL statements, you can create the skeleton of the database.

- Data definition language is used to store the information of metadata like the number of tables and schemas, their names, indexes, columns in each table, constraints, etc.

Tasks come under DDL :

- **Create:** It is used to create objects in the database.
- **Alter:** It is used to alter the structure of the database.
- **Drop:** It is used to delete objects from the database.
- **Truncate:** It is used to remove all records from a table.
- **Rename:** It is used to rename an object.
- **Comment:** It is used to comment on the data dictionary.

▼ **D**ata **M**anipulation **L**anguage (**DML**)

DML stands for **D** ata **M**anipulation **L**anguage. It is used for accessing and manipulating data in a database. It handles user requests.

Tasks come under the DML :

- **Select:** It is used to retrieve data from a database.
- **Insert:** It is used to insert data into a table.
- **Update:** It is used to update existing data within a table.
- **Delete:** It is used to delete all records from a table.
- **Merge:** It performs UPSERT operation, i.e., insert or update operations.
- **Call:** It is used to call a structured query language or a Java subprogram.
- **Explain Plan:** It has the parameter of explaining data.
- **Lock Table:** It controls concurrency.

▼ **D**ata **C**ontrol **L**anguage (**DCL**)

- **DCL** stands for **D**ata **C**ontrol **L**anguage. It is used to retrieve the stored or saved data.
- The DCL execution is transactional. It also has rollback parameters.

Task Comes under the DCL :

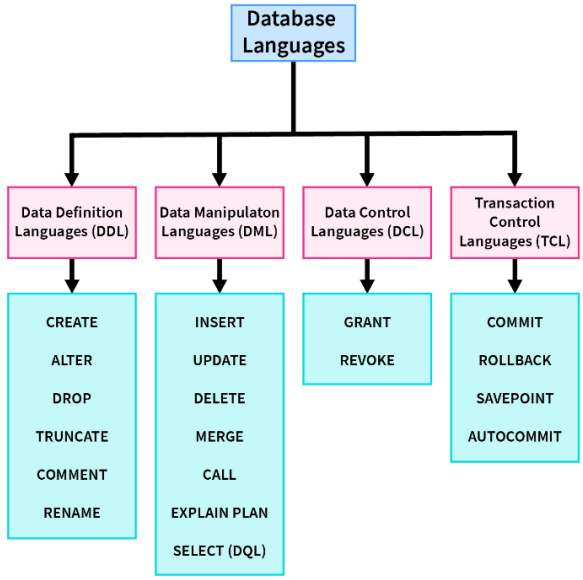
- **Grant:** It is used to give user access privileges to a database.
- **Revoke:** It is used to take back permissions from the user.

▼ **T**ransactional **C**ontrol **L**anguage(**TCL**)

TCL is used to run the changes made by the DML statement. TCL can be grouped into a logical transaction.

Task that come undet TCL

- **Commit:** It is used to save the transaction on the database.
- **Rollback:** It is used to restore the database to original since the last Commit.



▼ Database Design

Database design can be generally defined as a collection of tasks or processes that enhance the designing, development, implementation, and maintenance of enterprise data management system. Designing a proper database reduces the maintenance cost thereby improving data consistency and the cost-effective measures are greatly influenced in terms of disk storage space. Therefore, there has to be a brilliant concept of designing a database. The designer should follow the constraints and decide how the elements correlate and what kind of data must be stored.

Three phases of data base design :

1. **Conceptual database design**

Conceptual level (high level) data model is the data model that porvides concepts close to the way how users perceive (see) data, using this model a conceptual schema (structure) is created.

- This design deals with creating a conceptual schema.

```
Conceptual Schema →
Concise description of  data requirements & detaileddescription of the entity types,relationships & constraints.
```

- The design where the non-technical users could able to communicate with data base.

2. **Logical design**

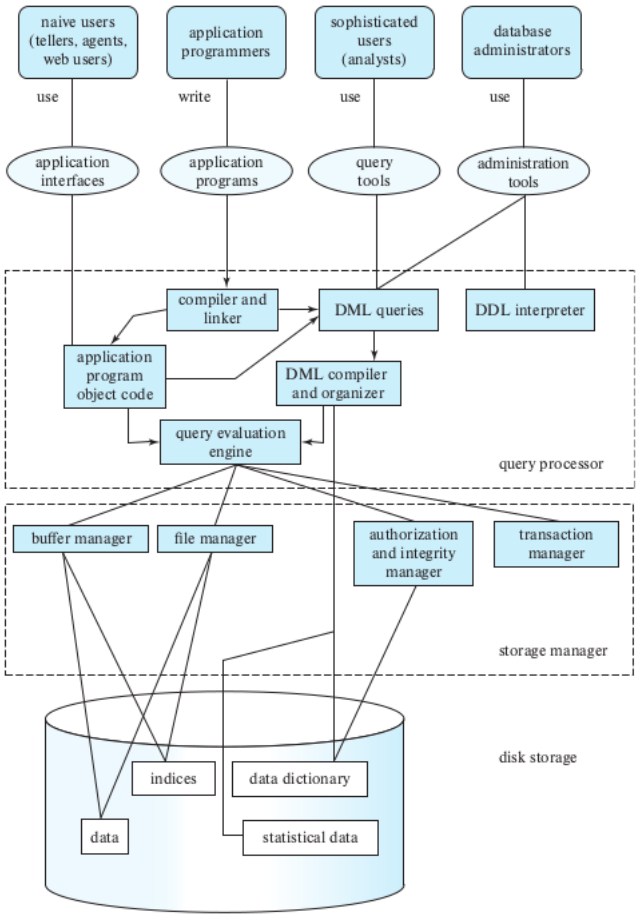
- Actual implementation of the databse, using commercial DBMS is happened here.
- Here the conceptual data model is converted to implementation data model.

3. **physical design**

- The internal storage structures,indexes,access paths are specifed.

▼ Database and Application Architecture

▼ Database Engine



The function of the storage manager is classified into 3 parts oriented mechanism, the parts of a database Engine are

▼ Views

▼ Naive Users

The users who query and update the database by invoking some already written application programs. For example, the owner of the bookstore enters the details of various books in the database by invoking an appropriate application program.

▼ Application programmers

Application Programmers also referred as System Analysts or simply Software Engineers, are the **back-end programmers who writes the code for the application programs**. They are the computer professionals.

▼ Sophisticated Users

Sophisticated users can be engineers, scientists, business analyst, who are familiar with the database. They can develop their own database applications according to their requirement. They don’t write the program code but they interact the database by writing SQL queries directly through the query processor.

▼ Database administrators

Database administrator(DBA)is a person controlling and coordinating the database management system and responsible for authorizing access to database, coordinating and monitoring it's use and for acquiring software and hardware resources as needed.

It's importance in DBMS are:-

1. The DBA administers the three levels of database like internal level,conceptual level and external level of DBMS architecture and in consultation with the overall user community, sets up the definition of global view or conceptual level of database. It further specifies the external view of various users and applications.
2. DBA ensures that appropriate measures are in place to maintain the integrity of database and that the database is not accessible to unauthorized users. It is responsible for granting permission to the users of the database and stores the profile of each user in database .
3. DBA is responsible for defining procedures to recover the database from failures due to human,natural or hardware causes minimal loss of data.

▼ Query Processor

- Processing queries written in the database's query language, such as SQL
- Converting queries into a form that the database can understand and execute
- Optimizing the execution of queries, including choosing the most efficient algorithms and data structures to use and determining the best order in which to execute the operations in the query
- Retrieving and manipulating data as needed
- Using techniques such as caching and indexing to improve the performance of queries.

▼ Storage Manager

- Helps to manage the data Storage.
- Provides interface b/w low level data stored in database and app programs along with the queries.
- Responsible for the interaction with the file manager.
- Translate various DML statements into low low level file system commands.
- Responsibilities are :
  - storing data
  - updating data
  - retrieving data

▼ Components of storage manager

▼ Authorization and Integrity Manager

- Enforcing access control policies to determine which users or applications are allowed to access which data

- Authenticating users and granting or revoking access to specific data or operations
- Maintaining the integrity of the data by enforcing constraints and preventing unauthorized changes
- Monitoring for and preventing any unauthorized changes to the data.

▼ **Transaction Manager**

- Coordinating the execution of transactions
- Ensuring that transactions are executed in a consistent, isolated, and durable way
- Maintaining the consistency of the data by ensuring that all operations within a transaction are either all committed or all rolled back
- Coordinating access to the data by multiple transactions, using techniques such as locking to prevent data corruption and ensure the integrity of the data.

▼ **File Manger**

- Managing the physical storage of the data on disk or other media
- Reading and writing data to and from the storage device
- Organizing the data in an efficient way
- Maintaining the integrity of the data, including handling errors and corruptions
- Optimizing the placement of data to improve performance.

▼ **Buffer Manager**

- Managing a memory buffer that is used to store data that has been read from or written to the physical storage device
- Reducing the number of disk accesses required to read and write data by storing data in the buffer in memory
- Deciding which data to store in the buffer and when to write data back to the storage device
- Optimizing the use of the buffer through techniques such as caching and prefetching.

▼ Disk Storage

The disk storage is managed by the storage manager and the file manager, which are components of the database engine.

common components of a disk storage in a database engine:

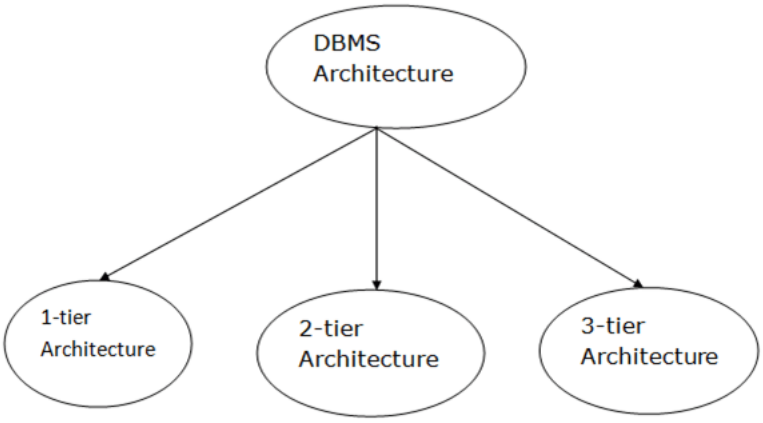
1. **Data files:** The data files are the actual files on the disk where the data is stored. These files are typically organized into tables, with each file representing a single table.
2. **Indexes:** Indexes are data structures that are used to improve the performance of queries by providing a faster way to locate data. Indexes are typically stored as separate files on the disk.
3. **Data Dictionary:** The data dictionary is a component of the disk storage in a database that stores metadata about the structure and organization of the data in the database. This metadata may include information about the tables, fields, and

relationships in the database, as well as other details about the data and how it is used.

4. **Statistical Data:** Statistical data is data that is used to describe the characteristics of a population or a sample. In a database engine, statistical data may be stored in the database itself, or it may be generated by analyzing the data in the database.

▼ Database Architecture

- The DBMS design depends upon its architecture. The basic client/server architecture is used to deal with a large number of PCs, web servers, database servers and other components that are connected with networks.
- The client/server architecture consists of many PCs and a workstation which are connected via the network.
- DBMS architecture depends upon how users are connected to the database to get their request done.



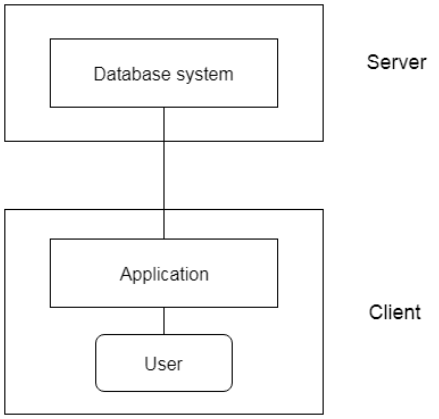
1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.

2-Tier Architecture

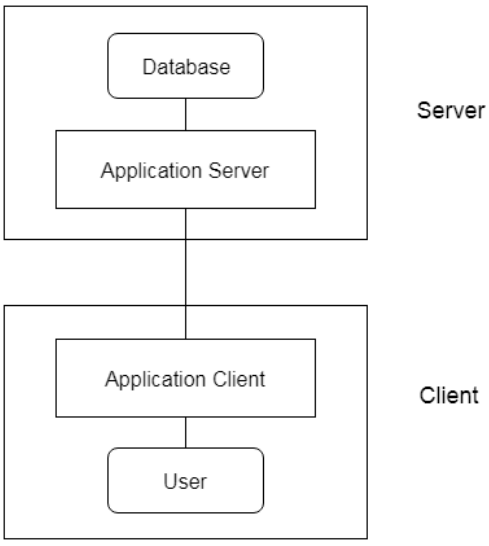
- The 2-Tier architecture is same as basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database at the server side. For this interaction, API's like: **ODBC**, **JDBC** are used.
- The user interfaces and application programs are run on the client-side.
- The server side is responsible to provide the functionalities like: query processing and transaction management.

- To communicate with the DBMS, client-side application establishes a connection with the server side.



3-Tier Architecture

- The 3-Tier architecture contains another layer between the client and server. In this architecture, client can't directly communicate with the server.
- The application on the client-end interacts with an application server which further communicates with the database system.
- End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- The 3-Tier architecture is used in case of large web application.



▼ Data Models

Data Model is the modeling of the data description, data semantics, and consistency constraints of the data. It provides the conceptual tools for describing the design of a database at each level of data abstraction. Therefore, there are following four data models used for understanding the structure of the database:

▼ Relational Data Model

This type of model designs the data in the form of rows and columns within a table. Thus, a relational model uses tables for representing data and in-between relationships. Tables are also called relations. This model was initially described by `Edgar F. Codd`, in 1969. The relational data model is the widely used model which is primarily used by commercial data processing applications.

eg:

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20
4	SURESH	DELHI		18

IMPORTANT TERMINOLOGIES

- **Attribute:** Attributes are the properties that define a relation. e.g.; **ROLL_NO**, **NAME**
- **Relation Schema:** A relation schema represents the name of the relation with its attributes. e.g.; STUDENT (ROLL_NO, NAME, ADDRESS, PHONE, and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation, it is called Relational Schema.
- **Tuple:** Each row in the relation is known as a tuple. The above relation contains 4 tuples, one of which is shown as:

1	RAM	DELHI	9455123451	18

- **Relation Instance:** The set of tuples of a relation at a particular instance of time is called a relation instance. Table 1 shows the relation instance of STUDENT at a particular time. It can change whenever there is an insertion, deletion, or update in the database.
- **Degree:** The number of attributes in the relation is known as the degree of the relation. The **STUDENT** relation defined above has degree 5.
- **Cardinality:** The number of tuples in a relation is known as cardinality.
  The **STUDENT** relation defined above has cardinality 4.
- **Column:** The column represents the set of values for a particular attribute. The column **ROLL_NO** is extracted from the relation STUDENT.

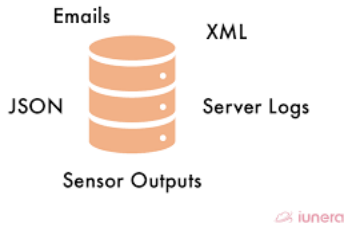
ROLL_NO
1
2
3
4

- **NULL Values:** The value which is not known or unavailable is called a NULL value. It is represented by blank space. e.g.; PHONE of STUDENT having ROLL_NO 4 is NULL.

▼ Semi-structured Data Model

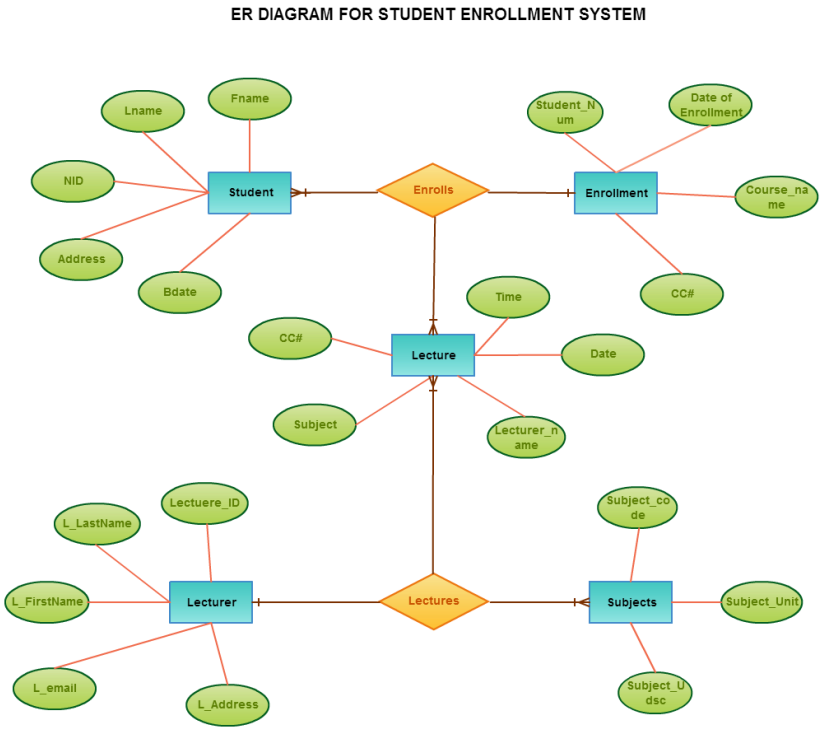
This type of data model is different from the other three data models . The semi-structured data model allows the data specifications at places where the individual data items of the same type may have different attributes sets. The Extensible Markup Language, also known as XML, is widely used for representing the semi-structured data. Although XML was initially

designed for including the markup information to the text document, it gains importance because of its application in the exchange of data.



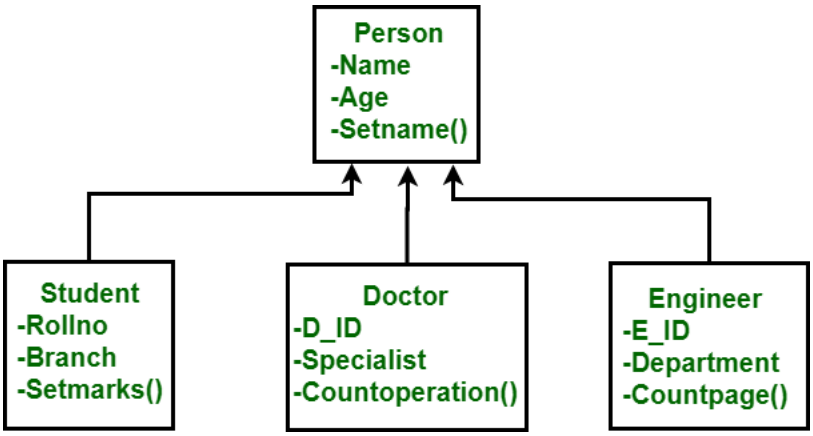
▼ Entity-Relationship Data Model

An ER model is the logical representation of data as objects and relationships among them. These objects are known as entities, and relationship is an association among these entities. This model was designed by Peter Chen and published in 1976 papers. It was widely used in database designing. A set of attributes describe the entities. For example, student_name, student_id describes the 'student' entity. A set of the same type of entities is known as an 'Entity set', and the set of the same type of relationships is known as 'relationship set'.



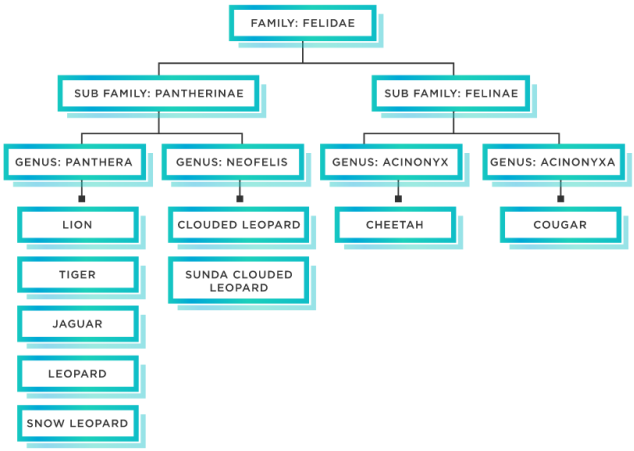
▼ Object-based Data Model

An extension of the ER model with notions of functions, encapsulation, and object identity, as well. This model supports a rich type system that includes structured and collection types. Thus, in 1980s, various database systems following the object-oriented approach were developed. Here, the objects are nothing but the data carrying its properties.



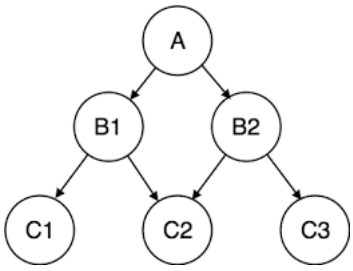
▼ Hierarchical Data Model

Hierarchical data is **a data structure when items are linked to each other in parent-child relationships in an overall tree structure**. Think of data like a family tree, with grandparents, parents, children, and grandchildren forming a hierarchy of connected data.



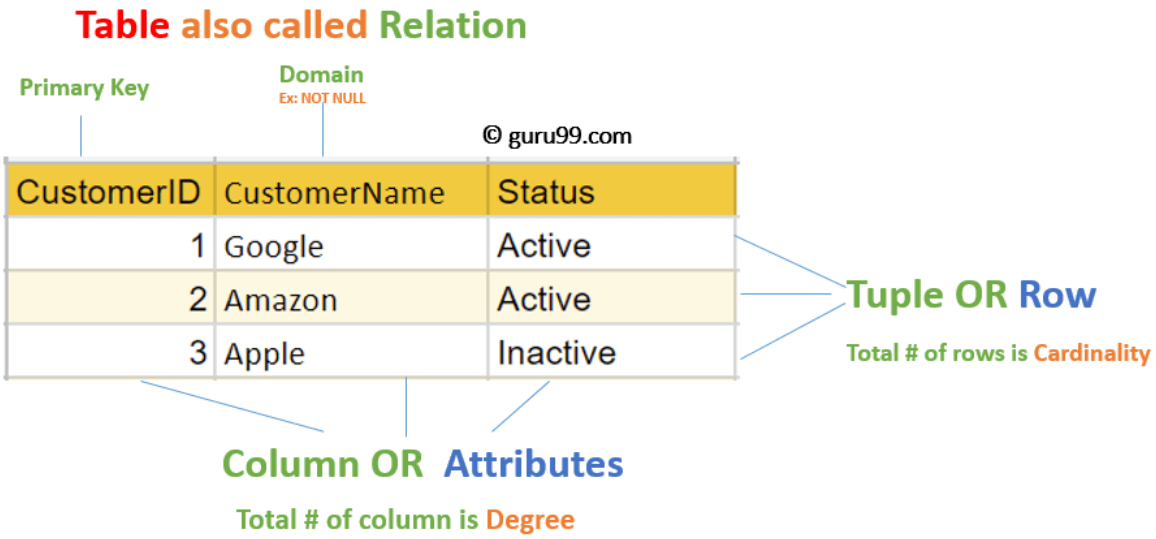
▼ Network Data Model

In this type of model, a child can be linked to multiple parents, a feature that was not supported by the hierarchical data model. The parent nodes are known as owners and the child nodes are called members.

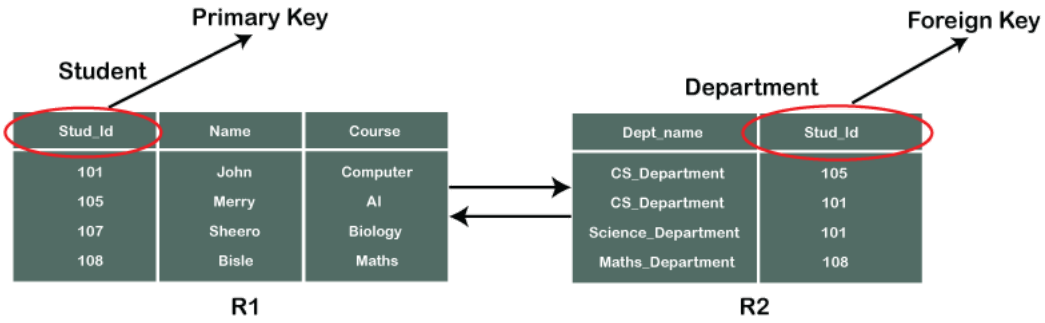


▼ Introduction to Relational Model

▼ Structure of relational database

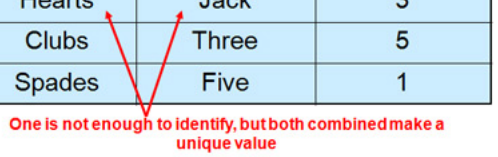


1. A relational database consists of a collection of tables. Each table is made up of rows (also known as records or tuples) and columns (also known as fields or attributes).
2. Tables in a relational database are related to one another through common fields, known as keys. There are several types of keys, including primary keys, foreign keys, and composite keys.



3. Primary keys are unique identifiers for each row in a table. They are used to link rows in different tables together and ensure the integrity of the data.
4. Foreign keys are fields that reference the primary key of another table. They are used to establish relationships between tables.
5. Composite keys are keys that are made up of more than one field. They are used when a table does not have a single field that can uniquely identify each row.

Suit	Value	No. times played
Hearts	Ace	5
Diamonds	Three	2
Hearts	Jack	3
Clubs	Three	5
Spades	Five	1



6. In a relational database, data is organized into logical entities called entities. An entity is a collection of related data that is stored in a table.
7. Relational databases use Structured Query Language (SQL) to create, modify, and query the data stored in the tables.

▼ Database schema

A database schema is the overall design of a database, including the structure and organization of the data. In a relational database, the schema defines the tables, fields, relationships, and other elements of the database.

Database instance

A database instance is a running instance of a database management system (DBMS). When a DBMS is installed on a computer, it creates a database instance that can be used to store and manage data.

▼ Keys

Topics mentioned in the textbook:

- *super key*
- *candidate key*
- *primary key*
- *Foreign key*

https://youtu.be/_UZLrD_R0T4

keys are fields that are used to identify and organize the data stored in the database. There are several types of keys, including primary keys, foreign keys,super key and composite keys.

▼ Super Key

A super-key is a set of one or more fields that can uniquely identify a row in a table. A super-key can include a primary key or a combination of fields that, taken together, are unique for each row.

▼ Example 1:

consider a table of employees with the following fields: Employee ID, First Name, Last Name, and Department. The Employee ID field could be used as the primary key,

since it is unique for each employee. However, a super-key could also be created using a combination of the First Name, Last Name, and Department fields. This super-key would be unique for each employee as long as there are no two employees with the same first and last name in the same department.

▼ Example 2:

Emp_SSN	Emp_Id	Emp_name	Emp_email
11051	01	John	john@email.com
19801	02	Merry	merry@email.com
19801	03	Riddle	riddle@email.com
41201	04	Cary	cary@email.com

Emp_SSN: The SSN number is stored in this field.

Emp_Id: An attribute that stores the value of the employee identification number.

Emp_name: An attribute that stores the name of the employee holding the specified employee id.

Emp_email: An attribute that stores the email id of the specified employees.

Set of super keys obtained

{ Emp_SSN }
{ Emp_Id }
{ Emp_email }
{ Emp_SSN, Emp_Id }
{ Emp_Id, Emp_name }
{ Emp_SSN, Emp_Id, Emp_email }
{ Emp_SSN, Emp_name, Emp_Id }

▼ Candidate key

A candidate key is a subset of a super key set where the key which contains no redundant(repetitive) attribute is none other than a **Candidate Key**.

A candidate key is a set of fields that meets the following criteria:

1. It is unique: Each row in the table has a different combination of values for the fields in the candidate key.
2. It is minimal: It contains the smallest number of fields necessary to ensure uniqueness.
3. It does not contain null values except for certain cases.

Example

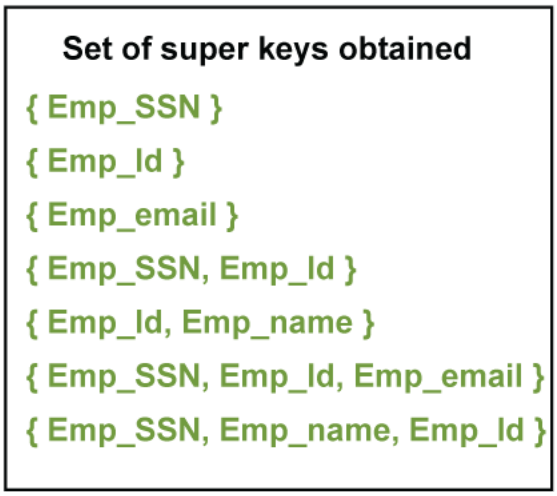
Emp_SSN	Emp_Id	Emp_name	Emp_email
11051	01	John	john@email.com
19801	02	Merry	merry@email.com
19801	03	Riddle	riddle@email.com
41201	04	Cary	cary@email.com

Emp_SSN: The SSN number is stored in this field.

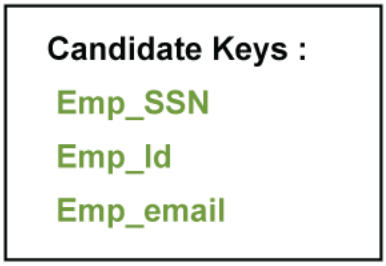
Emp_Id: An attribute that stores the value of the employee identification number.

Emp_name: An attribute that stores the name of the employee holding the specified employee id.

Emp_email: An attribute that stores the email id of the specified employees.



The candidate keys that are derived from the super keys mentioned here are :



▼ Primary Key

A primary key is a column or set of columns that uniquely identifies each row in a table,which is a subset of candidate key . The primary key can't contain null values and

must be unique across all rows in the table. It is used to ensure the integrity of the data in the table and to establish relationships with other table.

Example:

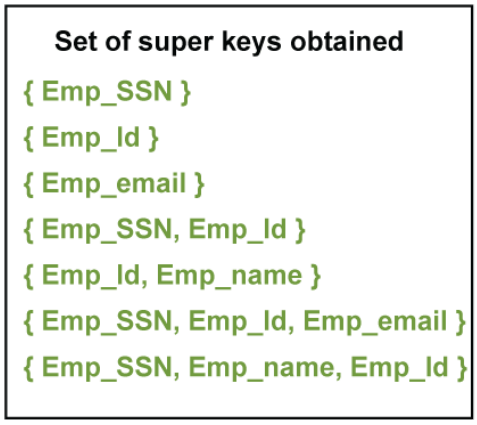
Emp_SSN	Emp_Id	Emp_name	Emp_email
11051	01	John	john@email.com
19801	02	Merry	merry@email.com
19801	03	Riddle	riddle@email.com
41201	04	Cary	cary@email.com

Emp_SSN: The SSN number is stored in this field.

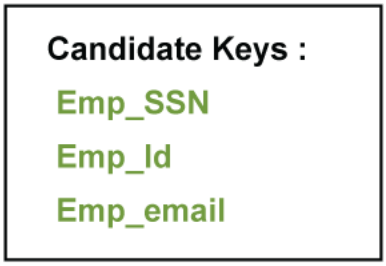
Emp_Id: An attribute that stores the value of the employee identification number.

Emp_name: An attribute that stores the name of the employee holding the specified employee id.

Emp_email: An attribute that stores the email id of the specified employees.



The candidate keys that are derived from the super keys mentioned here are :



The **Primary Key** from the above mentioned Candidate keys is

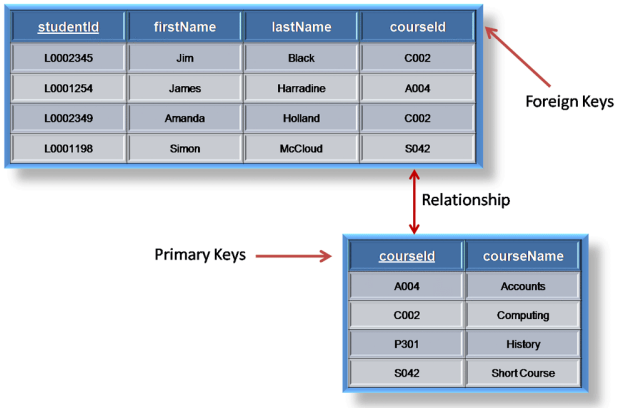
Primary key

- **Emp_Id**

Because of the reason that `Emp_id` could be duplicated in a schema, `Emp_id` is the primary key among the given data.

▼ Foreign Key

A foreign key is a column or set of columns that refers to the primary key of another table. The foreign key is used to establish a relationship between the two tables.A foreign key can also be used to enforce referential integrity, which means that if a row in the primary key table is deleted, any rows in the foreign key table that reference that primary key are also deleted.



▼ Difference B/W candidate key and Super key

Super Key	Candidate Key
It is the super-set of all such attributes that can uniquely identify the table.	It is the subset or the part of the Super key.
It is not at all compulsory that all super keys are candidate keys.	On the other hand, all candidate keys are super keys.
The **super key attribute can be NULL**, which means its values can be null.	An attribute holding a **candidate key can never be NULL**, which means its values cannot be null.
All the super keys formed together to bring the candidate keys.	Similarly, candidate keys are put together to create primary keys.
The number of super keys formed is always seen more.	Here, Candidate keys are less than super keys.

▼ Difference B/W candidate key and Primary key

S. No.	Primary Key	Candidate Key
1.	Primary key is a unique and non-null key which identify a record uniquely in table.	Candidate key is also a unique key to identify a record uniquely in a table.
2.	Primary key column value cannot be NULL.	Candidate key column can have NULL value in certain cases.
3.	Primary key is most important part of any relation or table.	Candidate key signifies as which key can be used as Primary Key.

S. No.	Primary Key	Candidate Key
4.	Primary Key is a candidate key.	Candidate key may or may not be a primary key.
5.	**A table can have only one primary key.**	**A table can have multiple candidate keys.**

▼ Relational Algebra and Calculus

Both **Relational Algebra** and **Relational Calculus** are formal query languages.

https://www.ccs.neu.edu/home/kathleen/classes/cs3200/4-RAAndRC.pdf

▼ Relational Algebra Operations

Relational Algebra is a procedural language which consists of a set of operations that take one or two relations as input and produce a new relation as their result.

▼ Unary Relational Operations

▼ Selection (σ)

- The selection is the

A selection operator `(σ)` is a unary operator in relational algebra. We use it to select the relation's records or rows that satisfy its condition(s).

Syntax:

$$\sigma_{\text{<selection\_condition>}}(R)$$

▼ Example 1

Select the EMPLOYEE tuples whose Department Number is 2 and Salary > 30000.

EMPLOYEE	FName	LName	Salary	DNo
	Tom	Ford	30000	3
	Amy	Jones	20000	2
	Alicia	Smith	40000	2
	Frank	Smith	38000	3

Input:

$$\sigma_{DNo=2 \text{ AND } Salary>30000}(EMPLOYEE)$$

Output:

FName	LName	Salary	DNo
Alicia	Smith	40000	2

▼ **Example 2**

Select the EMPLOYEE tuples who work in DNo = 3 and with Salary > 35000 or who work in DNo = 2 and with Salary > 25000.

EMPLOYEE	FName	LName	Salary	DNo
	Tom	Ford	30000	3
	Amy	Jones	20000	2
	Alicia	Smith	40000	2
	Frank	Smith	38000	3

Input

$$\sigma_{(DNo=3 \text{ AND } Salary>35000) \text{ OR } (DNo=2 \text{ AND } Salary>25000)}(EMPLOYEE)$$

Output

FName	LName	Salary	DNo
Alicia	Smith	40000	2
Frank	Smith	38000	3

▼ **Projection (Π)**

In relational algebra, the projection operation is used to select certain columns from a relation.

For example, if we have a relation with columns "A", "B", "C", and "D", the projection operation could be used to select only columns "B" and "D". The projection operation is typically denoted by the symbol π.

syntax :

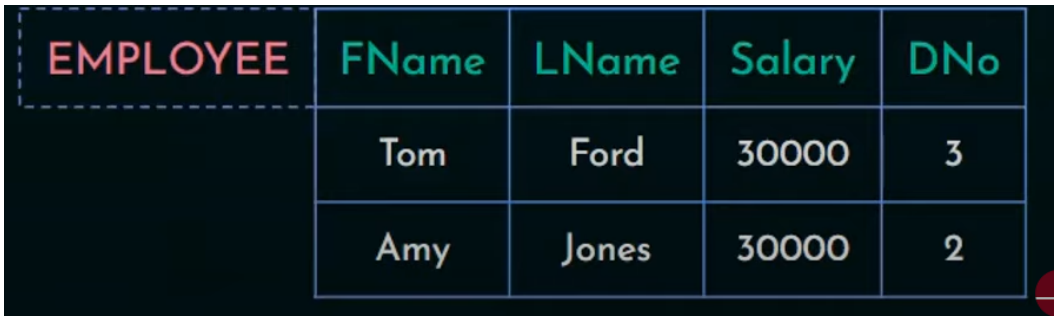
$$\pi_{<attribute\_list>}(R)$$

This would select columns "B" and "D" from relation R.

It's important to note that the projection operation does not modify the original relation. Instead, it creates a new relation that contains only the selected columns.

▼ **Example 1**

To list the Employee's First name, Last name and Salary.



EMPLOYEE	FName	LName	Salary	DNo
	Tom	Ford	30000	3
	Amy	Jones	30000	2

Input:

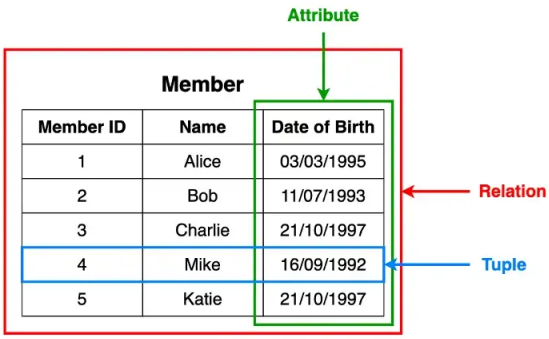
$$\pi_{Fname,\ LName,\ Salary}(EMPLOYEE)$$

Output:

FName	LName	Salary
Tom	Ford	30000
Amy	Jones	30000

▼ `Rename (ρ)`

The rename operation in relational algebra is used to change the name of a relation or attribute in a relation. It is typically denoted by the symbol ρ (rho).



syntax:

$$\rho_{S(B1, B2, \ldots, Bn)}(R)$$

R ⇒ Old Relation Name
S ⇒ New Relation Name
$B_1, \ldots, B_n$ ⇒ New Attribute Names

- This form is used to rename the relation name and attribute name

$$\rho_{S}(R)$$

- This form renames only the relation name not the attribute names.

$$\rho_{(B1, B2, \ldots, Bn)}(R)$$

- This form renames the attributes name,not the relation names.

▼ **Example 1:**

Rename the Member relation as LibraryMemeber.

Member

Member ID	Name	Date of Birth
1	Alice	03/03/1995
2	Bob	11/07/1993
3	Charlie	21/10/1997
4	Mike	16/09/1992
5	Katie	21/10/1997

Input

$$\rho_{LibraryMember}(Member)$$

Output

Member ID	Name	Date of Birth
1	Alice	03/03/1995
2	Bob	11/07/1993
3	Charlie	21/10/1997
4	Mike	16/09/1992
5	Katie	21/10/1997

Composite(Binary) Relational Operations

Cartesian product operation (X)

Cross product is used to combine data from two different relations into one combined relation. If we consider two relations; **A** with ***n*** tuples and **B** with ***m*** tuples, **A × B** will consist of ***n.m*** tuples.

Syntax:

`A × S`

where A and S are the relations,

the symbol '×' is used to denote the CROSS PRODUCT operator.

Example 1

Consider two relations STUDENT(SNO, FNAME, LNAME) and DETAIL(ROLLNO, AGE) below:

SNO	FNAME	LNAME
1	Albert	Singh
2	Nora	Fatehi

ROLLNO	AGE
5	18
9	21

Input

```
STUDENT × DETAILS
```

Output

SNO	FNAME	LNAME	ROLLNO	AGE
1	Albert	Singh	5	18
1	Albert	Singh	9	21
2	Nora	Fatehi	5	18
2	Nora	Fatehi	9	21

Join Operation(⋈)

A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied. It is denoted by ⋈.

Syntax

`A ⋈ B`

Where A and B are two related tuples from different relations

▼ **Example 1**

join the set of relations below :

EMPLOYEE

EMP_CODE	EMP_NAME
101	Stephan
102	Jack
103	Harry

SALARY

EMP_CODE	SALARY
101	50000
102	30000
103	25000

Input

EMPLOYEE ⋈ SALARY

Output

EMP_CODE	EMP_NAME	SALARY
101	Stephan	50000
102	Jack	30000
103	Harry	25000

▼ Set Relational Operations

▼ `Union (U)`

The union operation in Relational Algebra is very similar to that of set theory. However, for the union of two relations, both the relations must have the same set of attributes.

Syntax

`r U s`

Where **r** and **s** are either database relations or relation result set (temporary relation).

For a union operation to be valid, the following conditions must hold −

- **r**, and **s** must have the same number of attributes.
- Attribute domains must be compatible.
- Duplicate tuples are automatically eliminated.

▼ **Example 1**

Book IDs of the books borrowed by Charlie and Mike.

Member

Member ID	Name	Date of Birth
1	Alice	03/03/1995
2	Bob	11/07/1993
3	Charlie	21/10/1997
4	Mike	16/09/1992
5	Katie	21/10/1997

Book

Book ID	Name	Author
1	Inferno	Dan Brown
2	Ash	Malinda Lo
3	Fences	August Wilson
4	Origin	Dan Brown
5	Inheritance	Malinda Lo

Borrow

Member ID	Book ID	Burrow Date	Return Date
1	1	02/03/2020	12/03/2020
3	5	05/03/2020	15/03/2020
3	3	10/03/2020	20/03/2020
4	2	13/03/2020	23/03/2020
5	4	13/03/2020	13/03/2020

INPUT

$$R1 = \pi_{Book\ ID}(\sigma_{Name="Charlie"}(Member \bowtie Borrow))$$

$$R2 = \pi_{Book\ ID}(\sigma_{Name="Mike"}(Member \bowtie Borrow))$$

$$R1 \cup R2$$

OUTPUT

Book ID
2
3
5

▼ `Intersection (∩)`

It displays the common values in R1 & R2. It is denoted by ∩.(R1 nd R2 are the two relations or tables)

Syntax

`A ∩ B`

Where A , B are the two tables.

▼ **Example 1**

Depositor

ID	Name
1	A
2	B
3	C

Borrower

ID	Name
2	B
3	A
5	D

Find all the customers whose account is in the bank and have taken out a loan.

INPUT

`∏Name(Depositor) ∩ ∏Name(Borrower)`

OUTPUT

A
B

Set Difference (-)

The set-difference operation, denoted by −, allows us to find tuples that are in one relation but are not in another. The expression r − s produces a relation containing
those tuples in r but not in s.

Syntax

```
r - s
```

Example 1

Consider a relation Student(FIRST, LAST) and Faculty(FIRSTN, LASTN) given below :

Find the students exist in only the first table.

First	Last
Aisha	Arora
Bikash	Dutta
Makku	Singh
Raju	Chopra

FirstN	LastN
Raj	Kumar
Honey	Chand
Makku	Singh
Karan	Rao

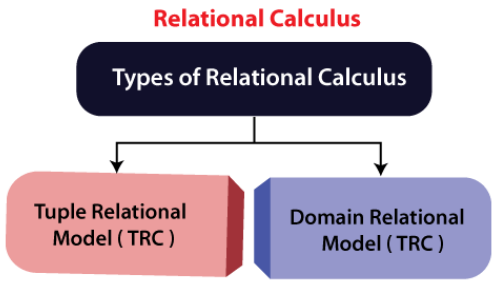
Input

```
Student - Faculty
```

Output

First	Last
Aisha	Arora
Bikash	Dutta
Raju	Chopra

Relational Calculus



Relational Calculus is the formal query language. It is also known as **Declarative language**. In Relational Calculus, the order is not specified in which the operation has to be performed. Relational Calculus means what result we have to obtain.

▼ Tuple Relational Calculus (TRC)

It is a non-procedural query language which is based on finding a number of tuple variables also known as range variable for which predicate holds true. It describes the desired information without giving a specific procedure for obtaining that information. The tuple relational calculus is specified to select the tuples in a relation. In TRC, filtering variable uses the tuples of a relation. The result of the relation can have one or more tuples.

Notation:

`{T | P (T)}`

`or`

`{T | Condition (T)}`

Where

T is the resulting tuples

P(T) is the condition used to fetch T.

Example 1:

`TRC` query that retrieves all tuples from the "Employees" relation where the "Age" attribute is greater than 30:

```
{t | t ∈ Employees ∧ t.Age > 30}
```

In this example, "t" is a variable representing a tuple in the "Employees" relation, and "t.Age" refers to the "Age" attribute of the tuple. The expression "t ∈ Employees" specifies that the tuple must be in the "Employees" relation, and the expression "t.Age > 30" specifies that the "Age" attribute of the tuple must be greater than 30.

Example 2:

write a `TRC` query that retrieves all tuples from the "Employees" relation where the "Department" attribute is "Sales" and the "Salary" attribute is greater than 50,000:

```
{t | t ∈ Employees ∧ t.Department = "Sales" ∧ t.Salary > 50000}
```

In this example, the expression "t.Department = "Sales"" specifies that the "Department" attribute of the tuple must be equal to "Sales", and the expression "t.Salary > 50000" specifies that the "Salary" attribute of the tuple must be greater than 50,000.

▼ Domain Relational Calculus (DRC)

The domain relational calculus is a formal language for specifying queries over the relationships defined in a database. It is a non-procedural language, meaning that you specify what you want to be retrieved from the database, but not how to retrieve it.

In `DRC`, queries are expressed in terms of variables and predicates over those variables. The variables represent the entities in the database, and the predicates specify the conditions that must be satisfied for a tuple (a row in the database) to be included in the output.

Expression Syntax

{<x1,x2,x3,x4...> \| P(x1,x2,x3,x4...)}

where,

<x1,x2,x3,x4...> are domain variables used to get the column values required, and **P(x1,x2,x3...)** is predicate expression or condition.

Example 1

consider a database with a relation (table) called "EMPLOYEE" with attributes (columns) "Name," "Department," and "Salary." To retrieve the names and salaries of all employees in the "Marketing" department, we could use the following DRC query:

```
{<Name, Salary> | EMPLOYEE(Name, Department, Salary) ^ (Department = "Marketing")}
```

Example 2

Retrieve the names and departments of all employees with a salary greater than $50,000:

```
{<Name, Department> | EMPLOYEE(Name, Department, Salary) ^ (Salary > 50000)}
```

Example 3

Retrieve the names and departments of all employees who work in the same department as "John Smith":

```
{<Name, Department> | EMPLOYEE(Name, Department, Salary) ^ (Name != "John Smith" and Department = (select Department from EMP
```